

CV-lab3 实验报告

杨子涵

2025 年 11 月 13 日

1 函数实现说明

1.1 Normalize_points 函数

把所有点归一化，使得中心位于原点，到原点的平均距离为 $\sqrt{2}$ 。实现的时候使用了 `np.mean()`, `np.sum()` 这些自带的函数，比 `for` 循环性能提升许多。

1.2 Fit_fundamental 函数

输入的是若干组 ground truth 的匹配点在图像坐标系下的坐标。已知 $(x_2, y_2, 1) \times F \times (x_1, y_1, 1)^T = 0$ ，目标是求解 F 。方法是把若干组 $(x_2, y_2, 1) \times F \times (x_1, y_1, 1)^T = 0$ 展开成一个线性方程组，每个匹配点贡献一个方程。

由于方程个数可能超过未知数个数，（设 f 为展平之后的 F ）所以目标由求解 $Af = 0$ 转化为最小化 $|Af|$ ，此时 f 应该取最小的特征向量。做法是标准的奇异值分解： $A = U\Sigma V^T$ ，则 V^T 的最后一行即为所求。

注：课件上使用了拉格朗日乘数法： $E = \|Af\|^2 + \lambda(\|f\|^2 - 1) = f^T A^T A f + \lambda f^T f - \lambda$ 。对 E 求导并令导数为零： $A^T A f = \lambda h, E = \lambda$ 因此 f 是 $A^T A$ 的特征向量，对应最小的特征值 λ 。但是我不矩阵求导。其实也可以利用线性代数的知识解决：我们要求解在 $\|f\|_2 = 1$ 的条件下 $\|Af\|$ 的最小值。由于 $A^T A$ 是一个实对称矩阵，它一定可以正交相似于一个对角矩阵 D ，即 $A^T A = P^T D P$ ，其中 D 的对角线上是 $A^T A$ 的特征值。因此：

$$\|Af\|^2 = (Af)^T (Af) = f^T A^T A f = f^T P^T D P f = (P f)^T D (P f)$$

令 $Pf = (y_1, \dots, y_n)^T$, 则 $y_1^2 + \dots + y_n^2 = (Pf)^T(Pf) = f^T P^T P f = 1$ 于是:

$$\|Af\|^2 = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \dots + \lambda_n y_n^2$$

在约束条件 $y_1^2 + y_2^2 + \dots + y_n^2 = 1$ 下, 该式的最小值等于最小的特征值。

代码中应注意 `np.linalg.svd` 的返回值是 U (矩阵), S (一个列表, 元素是奇异值), Vt (注意是 V 的转置)。由于 fundamental matrix 必须满足 $\text{rank}(F)=2$, 因此我们人为把求出来的最后一个奇异值设置为 0。

这里我补全一下课件上没有写出的证明过程, 感觉真的很巧妙。把最小的奇异值设置为 0 不仅满足 F 的秩为 2, 而且可以证明在所有秩为 2 的 F 里这样做可以最小化 $\|F - X\|$ (Frobenius 范数)。

首先有一个结论是对矩阵乘以正交矩阵之后范数不会改变, 从几何上来理解就是乘以正交矩阵相当于对向量做旋转, 不会改变向量的长度; 从代数上来理解只需注意到矩阵 M 的 Frobenius 范数的平方可以写成 $\text{trace}(M^T M)$ 。有了这样的理解之后我们开始我们的证明。

设 $F = U\Sigma V^T$, 其中 $\Sigma = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$ 且 $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$ 。我们要找的目标矩阵设为 X , 且 $\text{rank}(X) = 2$ 。则:

$$\|F - X\| = \|U\Sigma V^T - X\| = \|\Sigma - U^T X V\|$$

令 $Y = U^T X V$, 则 Y 也是秩为 2 的矩阵, 且:

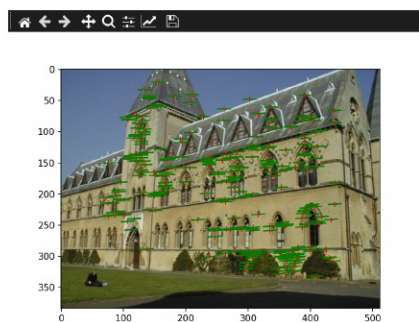
$$\|F - X\| = \|\Sigma - Y\|$$

由于 Σ 是对角矩阵, 而 Y 是任意秩为 2 的矩阵, 要使 $\|\Sigma - Y\|$ 最小, 这样可以很清晰地看出应该取 $\Sigma' = \text{diag}(\sigma_1, \sigma_2, 0)$, 此时:

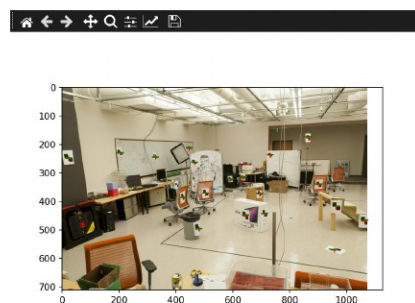
$$\min_{\text{rank}(X)=2} \|F - X\| = \|\Sigma - \Sigma'\| = \sigma_3$$

因此, 通过将最小的奇异值 σ_3 设置为 0, 我们得到了在所有秩为 2 的矩阵中最接近原始矩阵 F 的那个矩阵。

代码运行得到的可视化效果如下。图1a和图1b是未归一化的结果, 图2a和图2b是归一化的结果。放大之后可以看出 unnormalized 的可视化结果(图3a)里极线(绿线)并没有严格穿过匹配点(红色十字中心), 而 normalized 的可视化结果(图3b)里极线不偏不倚地经过了匹配点。

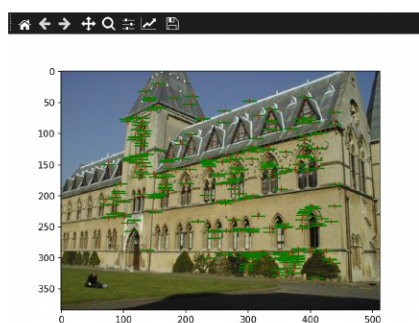


(a) 未归一化结果图一

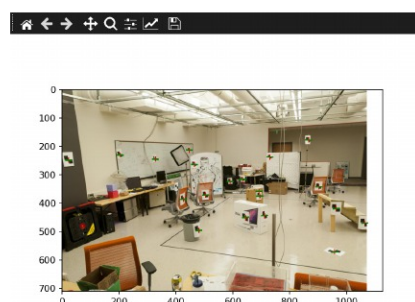


(b) 未归一化结果图二

图 1: 未归一化结果对比

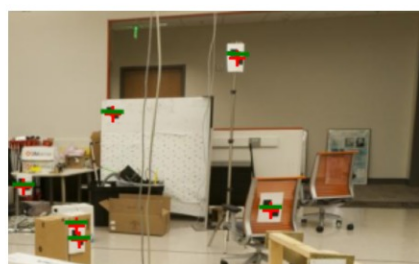


(a) 归一化结果图一

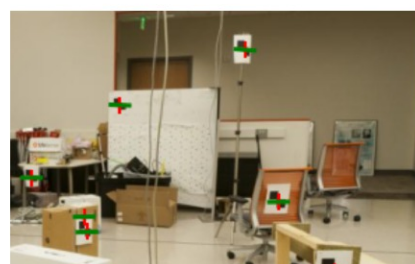


(b) 归一化结果图二

图 2: 归一化结果对比



(a) 未归一化结果放大图



(b) 归一化结果放大图

图 3: 放大细节对比

这是因为 normalized 可以提升数值稳定性，具体来说有如下两方面：

1. 输入的坐标是在图像坐标系下，不 normalize 的话不同点的坐标差距会很大，甚至差 2-3 个量级。求解 fundamental matrix 需要奇异值分解，而这对于误差也十分敏感。
2. 计算极线的方式是 $l = F \cdot x_1$ ，如果 F 有误差 ΔF ，则极线误差 $\Delta l = \Delta F \cdot x_1$ 。未归一化时 x_1 较大，导致误差 ΔF 被放大。

在终端输出 residual (图4) 可以看出确实 unnormalized 的 (上半部分) residual 要显著大于 normalized 之后的 residual (下半部分)，多出了 2-3 个数量级。

```
(cv) PS E:\PyCharm 2024.3.3\cv_hw3> python main.py
The value of evaluate_fundamental(library) is 0.0001614980653831963
The value of evaluate_fundamental(lab) is 0.0037634023769085056
Average residual for inliers: 0.102712
Average residual for inliers: 0.077555
(cv) PS E:\PyCharm 2024.3.3\cv_hw3> python main.py
The value of evaluate_fundamental(library) is 2.780328719287829e-06
The value of evaluate_fundamental(lab) is 7.394859404195483e-06
Average residual for inliers: 0.102712
Average residual for inliers: 0.077555
```

图 4: 残差对比图

1.3 calc_projection 函数

输入是一组点的图像坐标和三维坐标，输出是相机的参数矩阵。原理是 x_i (图像坐标), X_i (三维坐标), P (相机的参数矩阵) 应该满足关系 $x_i \simeq PX_i$ 。这个式子可以展开成一个线性方程 (未知数是 P 的 $3 \times 4 = 12$ 个元素)。每一组对应点都能提供两个方程，所以问题可以转化为求解 $Ap = 0$ 。

同样地，由于匹配点数可能大于未知数个数，所以我们的目标转化为最小化 $|Ap|$ ，于是我们用奇异值分解就能求出需要的 p 。

1.3.1 实验结果

对于 Lab 图像对的两个相机视角，投影矩阵估计结果如下：

- Camera A (lab_a): 投影残差 = 13.55，平均像素距离 = 3.97
- Camera B (lab_b): 投影残差 = 15.54，平均像素距离 = 3.86

两个相机的投影残差均小于 20，平均像素距离均小于 4，符合作业要求，表明投影矩阵估计准确。

1.4 rq_decomposition 函数

输入是 3×4 的矩阵 P ，输出是相机矩阵 K, R, T 。代码原理就是 $P = K[R|T]$ 。由于 K 是内参矩阵（上三角矩阵）， R 是三维坐标的旋转矩阵（正交矩阵），因此可以由 P 的前三列做 RQ 分解得到。

这里要注意由于 K 的对角线上元素为正数，因此在 `scipy.linalg.rq` 之后要检查对角线上元素的符号，如果符号为负则需要将 K 的这一列和 R 的这一行同时变号（保持 KR 不变的情况下把对角线元素变成正的，因为对角线代表焦距）然后对 K 进行归一化，可以让最右下角的元素为 1 或者矩阵范数为 1，不过标准的内参矩阵形式里应该是右下角元素为 1。

1.4.1 分解结果与分析

Lab 图像对相机参数

- Camera A (lab_a):

$$K = \begin{bmatrix} 780.57 & 1.99 & 545.67 \\ 0.00 & 779.99 & 384.16 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}, \quad R = \begin{bmatrix} 0.85 & -0.53 & -0.03 \\ -0.13 & -0.16 & -0.98 \\ 0.51 & 0.83 & -0.21 \end{bmatrix}, \quad T = \begin{bmatrix} -99.08 \\ 119.30 \\ -403.65 \end{bmatrix}$$

- Camera B (lab_b):

$$K = \begin{bmatrix} 767.02 & 8.37 & 536.21 \\ 0.00 & 772.02 & 390.71 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}, \quad R = \begin{bmatrix} 0.43 & -0.90 & -0.04 \\ -0.21 & -0.06 & -0.98 \\ 0.88 & 0.43 & -0.22 \end{bmatrix}, \quad T = \begin{bmatrix} 147.52 \\ 113.64 \\ -390.49 \end{bmatrix}$$

分析：

Lab 相机的内参矩阵 K 显示焦距较大（~770-780 像素），且存在轻微的 skew 项（1.99 和 8.37），表明每个像素实际经过了轻微拉伸，形状为平行四边形。

查阅 lab1.jpg, lab2.jpg 文件的属性可知 Lab 图像尺寸为 1072×712 像素，理论图像中心应为 (536.0, 356.0)。Camera A 的计算结果中 $(c_x, c_y) = (545.67, 384.16)$ ，与中心偏差 (9.67, 28.16) 像素；Camera B 的计算结果中 $(c_x, c_y) = (536.21, 390.71)$ ，与中心偏差 (0.21, 34.71) 像素。总得来说计算结果与理想情况较为符合，但是两幅图像计算出的 (c_x, c_y) 也有一定区别。这可能源于计算误差或镜头畸变。

Library 图像对相机参数

- **Camera A (library_a):**

$$K = \begin{bmatrix} 579.79 & 0.00 & 256.99 \\ 0.00 & 539.71 & 204.32 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}, \quad R = \begin{bmatrix} 0.01 & 0.44 & 0.90 \\ 0.98 & 0.17 & -0.10 \\ -0.20 & 0.88 & -0.43 \end{bmatrix}, \quad T = \begin{bmatrix} -6.49 \\ -1.71 \\ 28.03 \end{bmatrix}$$

- **Camera B (library_b):**

$$K = \begin{bmatrix} 547.47 & 0.00 & 258.43 \\ 0.00 & 512.93 & 204.99 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}, \quad R = \begin{bmatrix} -0.02 & 0.68 & 0.74 \\ 0.98 & 0.15 & -0.12 \\ -0.19 & 0.72 & -0.67 \end{bmatrix}, \quad T = \begin{bmatrix} -6.71 \\ -1.69 \\ 28.02 \end{bmatrix}$$

分析:

Library 相机的焦距较小 (~ 550 像素), 且无 skew 项, 表明相机传感器比较精确。

查阅 library1.jpg, library2.jpg 文件的属性可知 Library 图像尺寸为 512×384 像素, 理论图像中心应为 $(256.0, 192.0)$ 。Camera A 的计算结果中 $(c_x, c_y) = (256.99, 204.32)$ 与中心偏差 $(0.99, 12.32)$ 像素; Camera B 的计算结果中 $(c_x, c_y) = (258.43, 204.99)$ 与中心偏差 $(2.43, 12.99)$ 像素。计算结果与理想情况很符合, 而且两幅图像计算出的 (c_x, c_y) 十分一致, 再次说明了 library 图像使用的相机质量很高。

其他观察:

计算可得所有旋转矩阵 R 都满足正交性条件 ($RR^T \approx I$), 验证了 RQ 分解的正确性。

- **Lab Camera A:** $R \cdot R^T = \begin{bmatrix} 1.0043 & 0.0037 & -0.0001 \\ 0.0037 & 1.0029 & 0.0067 \\ -0.0001 & 0.0067 & 0.9931 \end{bmatrix}$

- **Lab Camera B:** $R \cdot R^T = \begin{bmatrix} 0.9965 & 0.0029 & 0.0002 \\ 0.0029 & 1.0081 & 0.0050 \\ 0.0002 & 0.0050 & 1.0077 \end{bmatrix}$

- **Library Camera A:** $R \cdot R^T = \begin{bmatrix} 1.0037 & -0.0054 & -0.0018 \\ -0.0054 & 0.9993 & -0.0034 \\ -0.0018 & -0.0034 & 0.9993 \end{bmatrix}$

- **Library Camera B:** $R \cdot R^T = \begin{bmatrix} 1.0104 & -0.0064 & -0.0024 \\ -0.0064 & 0.9973 & 0.0022 \\ -0.0024 & 0.0022 & 1.0034 \end{bmatrix}$

Lab 相机的平移向量 T 绝对值较大，表明相机距离场景中心较远，而 Library 相机的 T 值较小，表明相机更靠近场景。

1.5 triangulate_points 函数

输入是两个相机的参数矩阵和一组匹配点的二维坐标，输出是这组匹配点对应的三维坐标。原理依然是 $x_i \simeq PX_i$ ，这样两个点可以提供四个方程，而我们要求解的未知数有三个，方程个数超过未知数个数，因此同上使用奇异值分解即可。

1.6 fit_fundamental_without_gt

输入是两张图像，输出是 fundamental matrix 和匹配的特征点。在特征匹配阶段采用 cv2 内置的 SIFT 特征子，其效果明显优于我 Lab2 中自己实现的 HOG 特征子。Lab2 中我在自己数据集上测试的特征匹配结果如图5所示，即使两张图像存在大量重叠区域，由于光照和环境干扰，未能检测出有效的特征匹配。



图 5: Lab2 中 HOG 特征子匹配效果

采用 OpenCV 的暴力匹配器 (cv2.BFMatcher) 比较两幅图像提取的关键点特征：

```
knn_matches = bf.knnMatch(descriptors1, descriptors2, k=2)
```

设置 $k = 2$ 的想法是：仅依靠两个特征的最邻近距离不能保证匹配质量，只有当最近邻明显优于次近邻时才认为是可靠匹配。

获得初始匹配后，使用 RANSAC 进行鲁棒估计。在判断内点时，可采用三种距离度量：

- **Sampson 距离:** $D_{\text{Sampson}} = \frac{(\mathbf{x}_2^\top F \mathbf{x}_1)^2}{(F \mathbf{x}_1)_1^2 + (F \mathbf{x}_1)_2^2 + (F^\top \mathbf{x}_2)_1^2 + (F^\top \mathbf{x}_2)_2^2}$
- **Algebraic 距离:** $D_{\text{algebraic}} = |\mathbf{x}_2^\top F \mathbf{x}_1|$

- **Geometric 距离:** $D_{\text{geometric}} = d(\mathbf{x}_2, F\mathbf{x}_1)^2 + d(\mathbf{x}_1, F^\top \mathbf{x}_2)^2$

main.py 文件中的 fit_fundamental_without_gt 函数实现了这三种距离度量方法（其中两种被注释，一种保留使用）。此外，我让 AI 帮我写了测试文件 test_algorithm.py 评估三种算法的性能，终端输出结果如图6所示。

从输出结果可以看出在运行时间方面 Algebraic 距离最快，Sampson 距离次之，Geometric 距离最慢，不过总体数量级是一致的。

内点数量方面 Algebraic 距离最多，Sampson 距离次之，Geometric 距离最小。不过这并不能说明 Algebraic 距离最好，因为内点数量多不一定代表质量高，可能存在误匹配。

在 residual 方面 Geometric 距离最小，Sampson 距离次之，不过都显著小于 Algebraic 距离。这也说明 Algebraic 距离的内点质量很差。总的来说，Sampson 距离在速度和性能上做了一个很好的折中。

不过也应指出上述数据仅依赖于我在实验中选取的超参数（最关键的就是判断内点的 threshold）。对 Sampson 我选取的是 1，对 Algebraic 我选取的是 0.1，对 Geometric 我选取的是 2.0。这些值并不是最优的，可能通过调参可以提升某个算法的性能。

选取不同的超参数是因为这些距离的几何含义不同：Sampson 距离和 Geometric 距离都与像素距离相关，而 Algebraic 距离没有明确的几何意义，因此它的数量级会不同。没有明确的几何意义也正是 Algebraic 距离效果差的原因之一。

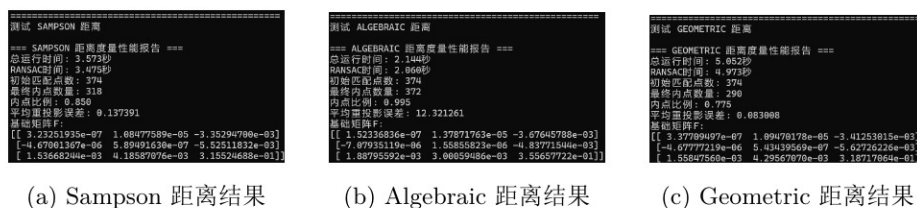


图 6: 三种距离度量算法的性能比较

使用三种不同距离的可视化结果如下图7所示。可以看出 Sampson 距离和 Geometric 距离可视化效果差距不大，都能较好地让极线穿过匹配点，而 Algebraic 距离的可视化结果中极线大部分偏离了匹配点，甚至有两处匹配点和在极线上的最近点相距甚远（图中明显的长红线）。

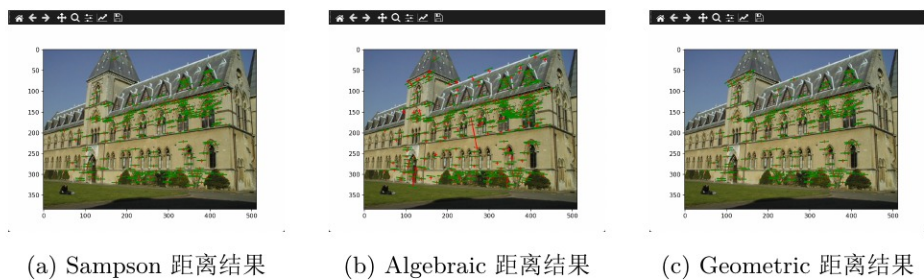


图 7: 三种距离度量算法的性能比较

最终我选择的实现方式是 Geometric 距离，可视化效果如下图8所示。

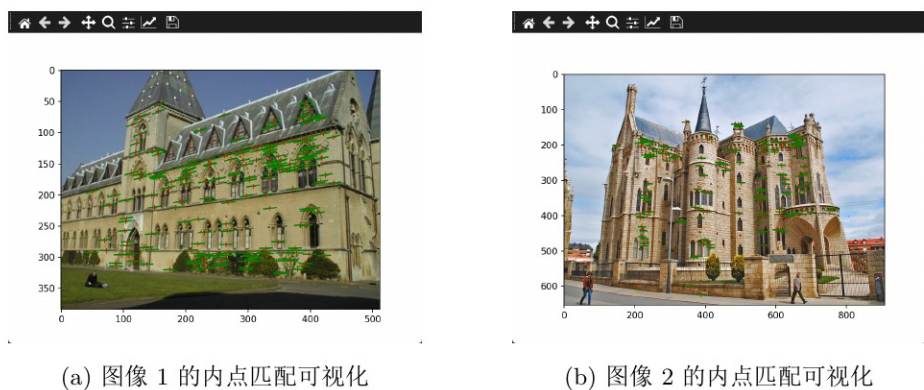


图 8: 基于 Geometric 距离的最终内点匹配可视化结果

在结果图中显示内点的结果如下图9所示。

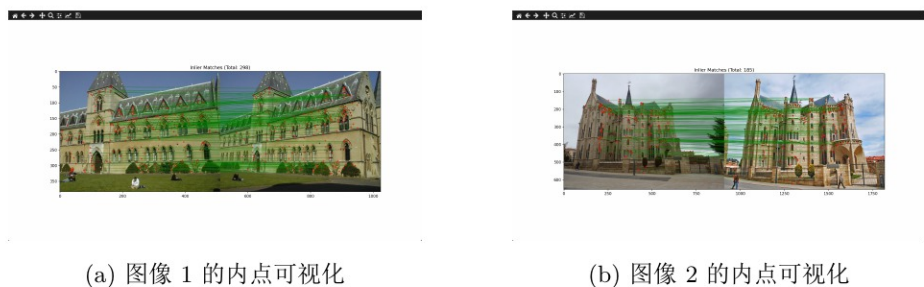


图 9: 最终内点可视化结果